UNIT - 3
Variables and Data Types

1. Fundamental of C

Program is a set of instructions to interact with computer. The programs are written in programing language. C is a procedural programming language developed at AT & T's Bell Laboratories of USA in 1972. It was designed and written by a man named Dennis Ritchie. C program is a sequence of characters that will be converted by the C compiler into machine code.

2. Character set of C

Character set is a package of valid characters recognized by compiler/interpreter. Each natural language has its own alphabet and characters set as the basic building. We combine different alphabets to make a word, sentence, paragraph and a meaningful passage. Similarly each computer languages have their own character set. Particularly, C also has its own character set, shows in following table:

Uppercase alphabets	A, B, C…….Z
Lowercase alphabets	a, b, c……z
Digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Special symbols	+ - * / ! ? | \ < > & , : ; # % * ( ) [ ] . _ $ ^
White space characters	Blank space, new line, horizontal tab, vertical tab, etc

3. Keywords

Keywords are the predefined reserved (built-in) words whose meaning has already been defined to the c compiler. A programmer can't use the keywords for other task than the task for which it has been defined. For example: a programmer can't use the keyword for variable names.

There are 32 keywords in c, which are listed in figure:

auto	do	goto	sizeof
break	double	if	static
case	else	int	switch
char	union	long	typedef
struct	enum	register	unsigned
const	extern	return	void
default	float	short	volatile
continue	for	signed	while

4. Identifiers

Identifiers are the user defined names and consists of a sequence of letters and digits with letter or underscore as a first character. It may be name of the variables, functions, arrays, etc. In identifiers we can use both upper and lower cases.

Rules for Identifiers

- First characters must be an alphabet or underscore.
- Must consist of only alphabets, digits or underscore.
- Cannot use a keyword.
- Must not contain white spaces. To connect two words underscore can be used.

5. Constants

Constant in C refers to fixed values that do not change during execution of a program. C supports several types of constants, such as numeric constants and character constants.

5.1 Numeric constants:

It consists of:

5.1.1 Integer Constants:

It refers to the sequence of digits with no decimal points. The three kinds of integer constants are:

- **Decimals**: Decimal numbers are set of digits from 0 to 9 with leading +ve or –ve sign. For example: 121, -512 etc.
- **Octals**: Octal numbers are any combination of digits from set 0 to 7 with leading 0. For example: 0121, 0375 etc.
- **Hexadecimals**: Hexadecimal numbers are the sequence of digits 0-9 and alphabets a-f (A-F) with preceding 0X or 0x. For example: 0xAB23, 0X787 etc.

5.1.2 Real or Floating point Constants:

They are the numeric constants with decimal points. The real constants are further categorized as:

- **Fractional Real constant:** They are the set of digits from 0 to 9 with decimal points. For example: 394.7867, 0.78676 etc.
- **Exponential Real constants:** In the exponential form, the constants are represented in two form: Mentissia E exponent; Where, the Mentissia is either integer or real constant but the exponent is always in integer form. For example: 21565.32 = 2.156532 E4, where E4 = 10^4.

5.2 Character constant:

Character constant are the set of alphabets. Every character has some integer value known as American Standard Code for Information Interchange (ASCII). The character constants are further categories as:

5.2.1 Single character constant:

It contains a single character enclosed with in a pair of single quote mark (' '). Thus, 'X', '5' are characters but X, 5 are not.

5.2.2 String constant:

String constants are a sequence of characters enclosed in double quotes marks (" "). They may be letters, numbers, special symbols or blank spaces. For example: "Hello", "X+Y". Note: 'x' ≠ "x" .

Characters	ASCII Values
A - Z	(65 – 90)
a - z	(97 – 122)
0 - 9	(48 – 57)

6. Delimiters

Delimiters are small system components that separate the various elements (variables, constants, statements) of a program. They are also known as separators. Some of the delimiters are:

- Comma: It is used to separate two or more variables, constants.
- Semicolon: It is used to indicate the end of a statement.
- Apostrophes (single quote): It is used to indicate character constant.
- Double quotes: It is used to indicate a string.
- White space: space, tab, new line etc.

7. Variables

A variable is a data name that may be used to store a data value. Unlike constants that remain unchanged during the execution of a program, a variable may take different values at different times during execution. Constant supplied by the user is stored in block of memory by the help of variable.

Consider an expression:

2*x + 3*y = 5. In this expression the quantities x and y are variable. A variable is a named location in memory that is used to hold certain values.

Rules for declaring variable name

1. Every variable name in C must start with a letter or underscore.
2. A valid variable name should not contain commas or blanks.
3. Valid variable name should not be keyword.
4. A variable name must be declared before using it.

Declaring variable

After defining suitable variable name, we must declare them before used in our program. The declaration deals with two things:

- It tells the compiler what the variable name is.
- It specifies what type of data the variable can hold.

Syntax:
Data_type variable_names;

For example:
int account_number;
float a, b, c;
char name[10];

8. Data Types

Data types are used to declare the variables and tell the compiler what types of data the variable will hold. Data type is useful in C that tells the computer about the type and nature of the value to be stored in a variable. The type of a variable determines how much space it occupies in storage and how the bit pattern stored is interpreted. Generally the data types can be categorized as:

8.1 Primary Data Types

These are the basic data types. There are three basic data types in C. They are integral data type(integer and constant fall under this type), real or floating data type and void type.

Data type	Description	Required memory
char	Character	1 Byte
int	Integer	2 Byte
float	Floating point number	4 Bytes
double	Double precision floating point number	8 Bytes
void	No value	0 bytes

8.2 Derived (or Secondary) Data Types

The secondary (derived) data types are a collection of primary (primitive) data types. These are derived from the primary data types. Example: array, union, structure, etc

8.3 User Defined Data Types

The data types defined by the user is known as user defined data types. Example: enum(enumerated data type) and typedef (type definition data type).

9. Operators

C supports a rich set of operators. An operator is a symbol that tells the computer to perform mathematical or logical operations on operands. Operators are used in programs to manipulate data. C operators can be classified into a number of categories:

9.1 Arithmetic operators:

Arithmetic operators are used for performing most common arithmetic (mathematical) operations. There are generally five arithmetic operators in C such as addition (+), subtraction (-), multiplication (*), division (/) and modulus division (%).

9.2 Logical operators

Expressions which use logical operators are evaluated to either true or false. There are generally three logical operators used in C such as Logical AND (&&), Logical OR (||) and Logical NOT (!).

9.3 Assignment Operator:

An assignment operator assign a value to a variable and is represented by "=" (equals to) symbol.

Syntax: Variable name = expression.
The Compound assignment operators (+=, *=, -=, /=. %=) are also used.

Operator	Name	Example
=	Assignment	C = A + B will assign value of A + B into C
+=	Add AND assignment	C += A is equivalent to C = C + A
-=	Subtract AND assignment	C -= A is equivalent to C = C - A
*=	Multiply AND assignment	C *= A is equivalent to C = C * A
/=	Divide AND assignment	C /= A is equivalent to C = C / A
%=	Modulus AND assignment	C %= A is equivalent to C = C % A

9.4Relational operators

These operators are used to form relational expressions, which evaluates to either true or false. (True - 1, false - 0). It is used for comparisons. C supports six relational operators.

Operator	Meaning
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to
==	is equal to
!=	is not equal to

9.5 Bitwise operators

These operators which are used for the manipulation of data at bit level are known as bitwise operators. These are used for testing the bits or shifting them right or left.

Operator	Meaning
&	Binary (bitwise) AND
|	Bitwise OR
^	Bitwise exclusive OR
<<	Bitwise left shift
>>	Bitwise right shift
~	Bitwise Ones Complement

9.6 Increment and Decrement Operators

The increment operator (++) increases its operand by 1 nad the decrement (--) decreases its operand by 1. They are the unary operators (i.e. they operate on a single operand). It can be written as:
x++ or ++x (post and pre increment)
x—or –x (post and pre decrement)

9.7Conditional Operator

The ternary operator pair ? and : are used to construct conditional operator. An expression that makes use of conditional operator is called conditional expression. Its general syntax is:
IfCondition? TRUE case : FALSE case
Example: x=(a>b)?a:b;

10. Type casting

When an operator is used to manipulate operands of different data type then each operand should have same data type to take operation because the final result must be in one data type. For example, the given expression is: z = int x + float y; then the data type of z must be only one. To handle such type of problems, the type conversion and type casting are introduced. The process of converting one data type into another type is known as type casting. C provides two types of type conversions:

10.1 Implicit type conversion:

In implicit type conversion, if the operands of an expression are of different types, the lower data type is automatically converted to the higher data type before the operation evaluation. The result of the expression will be of higher data type. it is the automatic type conversion done by the compiler.

10.2 Explicit type conversion:

In explicit type conversion, the user has to enforce the compiler to convert one data type to another data type by using typecasting operator. This method of typecasting is done by prefixing the variable name with the data type enclosed within parenthesis.

Syntax: (data type) variable/expression/value;

Example:

```
float sum;
Sum = (int) (1.5 * 3.8);
```

The typecast (int) tells the C compiler to interpret the result of (1.5 * 3.8) as the integer 5, instead of 5.7. Then, 5.0 will be stored in sum, because the variable sum is of type float.

11. The first C program

```
/* A program to print a message */   //comment
#include<stdio.h>                    //preprocessor directive
void main()                          //beginning of a program
{
printf("Hello World");               //print function
}                                    //end of a program
```

Output: Hello World

11.1 Comment:

A comment is a descriptive sentence written in a program. Although a comment is not compulsory in a program but it is good practice as it makes a program simpler to understand the logic, more readable, and correcting the future errors. A comments are not executable statements therefore the compilers ignore the comments. Comments in C can be written in two ways:

11.1.1 Single line comment: The comment that starts with // is known as single line comments.

11.1.2 Multiline comment: The comment that starts with /* and end with */ is known as multiline comment.

11.2 Preprocessor directive

The C Preprocessor is not part of the compiler, but is a separate step in the compilation process. In simplistic terms, a C Preprocessor is just a text substitution tool and they instruct compiler to do required pre-processing before actual compilation. All preprocessor commands begin with a pound symbol (#). A preprocessor is built into the C compiler. When a program is to be compiled, then the source code is first preprocessed. A preprocessor has following format:

#include<header file>

Thus, the preprocessor directive line must not end with any symbol and only one preprocessor directive can appear in one line.

Header File in C :

Header file contains different predefined functions, which are required to run the program. All header files should be included explicitly before main ( ) function. It allows programmers to separate functions of a program into reusable code or file. Header file has extension like '*.h'. The prototypes of library functions are gathered together into various categories and stored in header files. Various header files with their functions are:

- **stdio.h** (standard input output header file) used for standard function like
  - **printf()** - output function used to print a message or value of a variable on the screen.
  - **scanf()** - input function used to input a value through the keyboard.
  - **putchar**() - output function used to print a single character
  - **getchar()** - input function used to input a single character
  - **fprintf()** - output function used to print a file
  - **fscanf()** - input function used to input a file
- **string.h:** Includes the string functions like

puts() -output function used to print a string
gets() - input function used to input a string
strlen() - find the length of given sting
strcpy() - cppy string from variable to another

- **math.h:** Includes the mathematical functions like
  sqrt() - to find the square root of a number
  pow() - to calculate a value raised to power
  abs() - to find the absolute value of an integer
  log() - to calculate the natural logarithm, etc.
- **conio.h** (console input output header file) used for display controlling function like
  getch(); - Function used to hold the output value of program until pressing any key. It doesn't echo character.
  getche(); - It echoes (show)the character.
  clrscr(); - Function used to clear the screen holding the previous values.

11.3 main( )

C program may consist of more than one function. If a program consists only one function it must be main () that shows the beginning of a program. It is not possible to write a program without main () because program execution always starts with this function. The main ( ) is not followed by comma or semicolon.
The function main ( ) encloses all the statements within a pairs of braces {…..}. Each line written between the braces is a statement. Every statement is terminated by semicolon. The semicolon at the end of a statement separates one statement from other. More than one statement can be written on the same line by separating them with individual semi colon. However, it is good practice to write one statement per line.

11.4 Escape Sequences (Backslash Character Constants)

An escape sequence always begins with a backward slash and is followed by one or more special characters.

Escape sequence	Description
\' (single quote)	Output the single quote (') character.
\" (double quote)	Output the double quote (") character.
\? (question mark)	Output the question mark (?) character.
\\ (backslash)	Output the backslash (\) character.
\a (alert or bell)	Cause an audible (bell) or visual alert.
\b (backspace)	Move the cursor back one position on the current line.
\n (newline)	Move the cursor to the beginning of the next line.
\r (carriage return)	Move the cursor to the beginning of the current line.
\t (horizontal tab)	Move the cursor to the next horizontal tab position.

11.5 Input and Output Build-In Functions

11.5.1 Unformatted I/O statements

In the unformatted I/O function, we do not need to specify the format of data used by the variable. The C-compiler automatically knows the format according to the function name. Generally, the unformatted I/O functions are of char or string type. the various unformatted I/O are:

- **getchar()**: Input single character **Syntax**: char-variable=getchar();
- **putchar()**: Output single character **Syntax**:putchar(char-variable);

The getchar() and putchar() functions are defined on the stdio.h header file of C

Example:

```
char ch;
ch  = getchar();
putchar(ch);
```

- **gets()**: Input a string **Syntax**: gets(stringvariable);
- **puts()**: Output a string **Syntax**: puts(str);
- The gets() and puts() functions are defined on the string.h header file of C

Example:

```
char ch[5];
gets(ch);
```

puts(ch);

11.5.2 Formatted I/O Statements

C has a special formatting character (%) to arrange the input data in a particular format. Some of the format specifies are: %c - character, %d - integer, %f - float, %s - string, %ld - long integer, %lf- long double, etc. some of the formatted I/O are:

- **scanf():** scanf () function is used to read or input a value and store it to a variable. Address operator (&) is used before the variables which store the value in the specified variable name.

Syntax: scanf ("format string", &list of variables);
Example:
scanf ("%c %d %f", &ch, &i, &x);
scanf ("%2d%5d",&a,&b); /*if the input is 12345 & 10, a=12 & b=345 and if the input is 12 & 3456, a= 12 & b=3456*/
Note: If we put: scanf("%c,%d,%f", &ch, &i, &x); Then give the input with comma, for example: A, 6, 3.99

- **printf()**: The library function printf () is used to output the values. This function can be used in variety of form as described below:
  - **Character that will be printed on the screen as they appeared**

Syntax: printf ("control string");
Example: printf ("Enter name of student");

- **Format specification that define the output format for display of each item**

Syntax: printf ("control string", arg1, ..argn);
Example: printf ("%d", a);

- **Escape sequence character such as new line (\n), tab (\t) bell (\b) etc.**

Example: printf ("\t");

LAB SECTION

1. WAP to display the text "Hello World" on the screen.

```
#include<stdio.h>
#include<conio.h>
void main()
{
printf("Hello World");
getch();
}
```

2. WAP to display a inputted character and a string on the screen.

```
//For character printing
#include<stdio.h>
#include<conio.h>
void main()
{
char ch;
ch  = getchar();
putchar(ch);
getch();
}
```

```
//For string printing
#include<stdio.h>
#include<conio.h>
void main()
{
char ch[5];
gets(ch);
puts(ch);
getch();
}
```

3. WAP to convert pound into kilogram.

```
#include<stdio.h>
```

```
#include<conio.h>
void main()
{
clrscr();
float kg, pound;
printf("ENTER THE VALUE OF POUND:");
scanf("%f",&pound);
kg = 0.4536*pound;
printf("THE EQUIVALENT KG IS : %f", kg);
getch();.
}
```

4. Program to add any two numbers.

```
#include <stdio.h>
void main ( )
{
 int integer1; // first number to be entered by user
 int integer2; // second number to be entered by user
 int sum; // variable in which sum will be stored
 printf( "Enter first integer\n" ); // prompt
 scanf( "%d", &integer1 ); // read an integer
 printf( "Enter second integer\n" ); // prompt
 scanf( "%d", &integer2 ); // read an integer
 sum = integer1 + integer2; // assign total to sum
 printf( "Sum is %d\n", sum ); // print sum
 getch();
 clrscr();
 } // end function main
```

5. WAP to identify the given number is either even or odd.

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a;
printf("Please enter a number:");
scanf("%d", &a);
a = a%2;
if(a==0)
{
printf("The entered number is even");
}
if(a==1)
{
printf("The entered number is odd");
}
getch();
clrscr();
}
```

6. If a number is input through the keyboard. WAP to find its square root.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main( )
{
```

```
int n, z;
clrscr( );
printf("enter a number");
scanf("%d", &n);
z=sqrt(n);
printf("\nthe square root of  %d is  %d", n, z);
getch( );
}
```

7. Three quantities principle, time and rate are input through the keyboard. WAP to find simple interest.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
float p, t, r, si;
clrscr( );
printf("enter the principle, time and rate");
scanf("%f%f%f", &p, &t, &r);
si=(p*t*r)/100;
printf("\n the simple interest=%f ",si);
getch( );
}
```

8. Two numbers are input through the keyboard. WAP to find addition, subtraction, multiplication, quotient and remainder after division.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
int x, y, z1, z2, z3, z4, z5;
clrscr( );
printf("Enter two numbers");
scanf("%d  %d", &x, &y);
z1=x+y;
z2=x-y;
z3=x*y;
z4=x/y;
z5=x%y;
printf("summation=%d, difference=%d, multiplication=%d, quotient=%d and remainder=%d", z1, z2, z3, z4, z5);
getch( );
}
```

9. Temperature of a city in centigrade is input through the keyboard. WAP to convert this temperature into Fahrenheit degree.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
float c, f;
clrscr( );
printf("enter the temperature in centigrade"); scanf("%f", &c);
f=1.8*c+32;
printf("the temperature in Fahrenheit  =  %f", f);
getch( );
}
```

10. If the marks obtained by a student in 5 different subjects are input through the keyboard. WAP to find his/her total and percentage mark.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
int m1, m2, m3, m4, m5, total, per;
clrscr( );
printf("enter marks");
scanf("%f%f%f%f%f", &m1, &m2, &m3, &m4, &m5);
total=m1+m2+m3+m4+m5;
per=total/5;
printf("total marks=%f and percentage=%f", total, per);
getch( );
}
```

11. WAP to find the area & perimeter of a rectangle and area & circumference of a circle.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>/*for function pow( )*/
void main( )
{
float l, b, area1, perimeter, rad, area2, cir;
clrscr( );
printf("Enter length, breadth of a rectangle:");
scanf("%f%f", &l, &b);
printf("Enter radius of a circle:");
scanf("%f", &rad);
area1 = l*b;
perimeter = 2*(l+b);
area2 = 3.14*rad*rad;
cir = 2*3.14*rad;
printf("\nAarea of rectangle= %f \n",area1);
printf(" Perimeter of rectangle = %f", perimeter);
printf("\nArea of circle = %f", area2);
printf("\nCircumference of circle = %f", cir);
getch();
}
```

12. If a 4 digit number is input through the keyboard. WAP to find reverse.

```
#include<stdio.h>
#include<conio.h>
void main( )
{
int n, d1, d2, d3, d4, rev=0;
clrscr( );
printf("Enter a 4 digit number");
 scanf("%d", &n);
d4=n%10;
rev=rev+d4*1000;
n=n/10;
d3=n%10;
rev=rev+d3*100;
n=n/10;
d2=n%10;
```

```
rev=rev+d2*10;
n=n/10;
d1=n%10;
rev=rev+d1;
printf("reverse=%d", rev);
getch( );
}
```

Alternate method

```
#include<stdio.h>
#include<conio.h>
void main( )
{
int d1,d2,d3,d4,n;
printf("Enter a  4 digit number");
scanf("%d", &n);
d4=n%10;
n=n/10;
d3=n%10;
n=n/10;
d2=n%10;
n=n/10;
d1=n%10;
printf("reverse=%d%d%d%d", d4,d3,d2,d1);
getch( );
}
```

13. Write a Program to find the area of triangle when three sides of a triangle are given.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
void main()
{
float a,b,c,s,area;
printf  ("\nENTER THE THREE SIDES OF A TRINAGLE:");
scanf  ("%f%f%f",&a,&b,&c);
s = (a+b+c)/2;
area  = sqrt((s*  (s-a)*(s-b)*(s-c)));
printf("\n Sides of triangle are:  \na=%f\nb=%f\nc=%f",a,b,c);
printf("\n The area of triangle is:%f ",area);
getch();
}
```